

Shared by sanjana. This is a copy of a chat between Claude and sanjana. Content may include unverified or unsafe content that does not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

Files hidden
when shared



Here is my game_engine.py. When the player wins (e.g., Rock vs Scissors), the CPU gets the point. Find the bug and give me the minimal fix.

The bug is in the `rules` dictionary inside `determine_winner`. The keys are `(player, cpu)` tuples, but the winners are labeled backwards. For example, `("ROCK", "SCISSORS")` means the player picked Rock and the CPU picked Scissors, which is a player win, yet it maps to `"CPU"`.

Minimal fix: swap the values so the first three entries map to `"PLAYER"` and the last three to `"CPU"`:

```
python

rules = {
    ("ROCK", "SCISSORS"): "PLAYER",
    ("SCISSORS", "PAPER"): "PLAYER",
    ("PAPER", "ROCK"): "PLAYER",
    ("SCISSORS", "ROCK"): "CPU",
    ("PAPER", "SCISSORS"): "CPU",
    ("ROCK", "PAPER"): "CPU",
}
```

Nothing else needs to change. `play_round` already handles `"PLAYER"` and `"CPU"` correctly (score increments, result text, color), so once the lookup is right, the scores and messages will follow.

For your commit message, something like `Fix inverted win conditions in determine_winner` would describe it well.

Add a first-to-X match system (X = 5, as a constant). When someone reaches X, stop accepting button clicks, show a victory banner saying who won, and show "Press R to restart". R should reset scores and state. Give me only the changed parts of game_engine.py.

Here are the changed parts only.

1. Constant (top of file, below the imports):

```
python

WIN_SCORE = 5
```

2. In `__init__` (add after `self.showing_result = False`):

```
python

    self.match_over = False
    self.winner_text = ""
```

3. New method (add anywhere in the class):

```
python

def reset(self):
    self.player_choice = None
    self.cpu_choice = None
    self.player_score = 0
    self.cpu_score = 0
    self.result_text = "Make your move!"
    self.result_color = (220, 225, 235)
    self.showing_result = False
    self.match_over = False
    self.winner_text = ""
```

4. In `play_round` (add at the very end):

```
python

    if self.player_score >= WIN_SCORE or self.cpu_score >= WIN_SCORE:
        self.match_over = True
        self.winner_text = "YOU WIN THE MATCH!" if self.player_score >= WIN_SCORE else
```

5. Replace `handle_event`:

python

```
def handle_event(self, event):
    if self.match_over:
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
            return

    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        for btn in self.buttons:
            if btn.contains(event.pos):
                self.play_round(btn.choice_name)
                break
```

6. In `update` (add as the first line, so the final round's picks and result stay on screen):

python

```
if self.match_over:
    return
```

7. Replace the button loop at the end of `render`:

python

```
if self.match_over:
    color = (80, 230, 120) if self.player_score >= WIN_SCORE else (240, 80, 80)
    banner = self.font_title.render(self.winner_text, True, color)
    hint = self.font_arena.render("Press R to restart", True, (225, 225, 230))
    screen.blit(banner, (self.width // 2 - banner.get_width() // 2, self.height // 2 - banner.get_height() // 2))
    screen.blit(hint, (self.width // 2 - hint.get_width() // 2, self.height // 2 - hint.get_height() // 2))
else:
    for btn in self.buttons:
        btn.render(screen)
```

The buttons are hidden while the match is over, which frees their area for the banner and prevents overlap. This assumes your main loop passes every event, including `KEYDOWN`, to `engine.handle_event(event)`. If it only forwards mouse events, you'll need to forward key events too.

Make the CPU adaptive. Keep a history of the player's last ~5 throws. If the player repeatedly favors one move, bias the CPU (e.g., 60-70% probability) toward the counter-move, otherwise stay random. Don't make it unbeatable. Reset the history on restart.

Here are the changed parts only.

1. Constants (below `WIN_SCORE`):

python

```
HISTORY_LEN = 5          # how many recent player throws to remember
FAVOR_THRESHOLD = 3      # same move this many times in the history counts as "favoring"
COUNTER_PROB = 0.65      # chance the CPU plays the counter once a favorite is detected
COUNTERS = {"ROCK": "PAPER", "PAPER": "SCISSORS", "SCISSORS": "ROCK"}
```

2. In `__init__` (add after `self.cpu_score = 0`):

python

```
self.history = []
```

3. New method:

python

```
def choose_cpu_move(self):
    if len(self.history) >= FAVOR_THRESHOLD:
        favorite = max(self.choices, key=self.history.count)
        if self.history.count(favorite) >= FAVOR_THRESHOLD:
            counter = COUNTERS[favorite]
            if random.random() < COUNTER_PROB:
                return counter
        return random.choice([c for c in self.choices if c != counter])
    return random.choice(self.choices)
```

4. In `play_round`, replace the `self.cpu_choice = ...` line with:

python

```
self.cpu_choice = self.choose_cpu_move()
self.history.append(choice)
self.history = self.history[-HISTORY_LEN:]
```

5. In `reset` (add one line):

python

`self.history = []`**Notes:**

- The CPU picks its move before the current throw is added to the history, so it never sees the move you're making this round.
- With 5 throws and a threshold of 3, at most one move can qualify as the favorite, so there are no ties to handle.
- When the bias doesn't fire, the CPU picks randomly from the two non-counter moves, so the counter probability is exactly 65%. The CPU stays beatable. Mix up your throws or break a streak, and it falls back to pure random.
- To tune difficulty, raise or lower `COUNTER_PROB` (0.6 to 0.7 as you suggested), or change `FAVOR_THRESHOLD` (4 makes it slower to react).

Replace the text picks with procedurally drawn icons for Rock, Paper, and Scissors using `pygame.draw` only (no image files). Add a 3-2-1 countdown or shaking-fist animation before both choices are revealed. Block clicks during the animation, and make sure the 1.8s result timer starts after the reveal.

Here are the changed parts only.

1. Imports and constants (top of file):

python

`import math`

python

```
COUNTDOWN_STEP = 400          # ms per "3", "2", "1"
ANIM_DURATION = 3 * COUNTDOWN_STEP  # total pre-reveal animation time
```

2. Icon drawing (module level, below the constants and above `class GameEngine`):

python

```

def draw_rock(screen, cx, cy, s):
    pts = [(-0.45, 0.10), (-0.30, -0.30), (0.05, -0.45), (0.40, -0.25),
            (0.48, 0.15), (0.25, 0.42), (-0.20, 0.40)]
    poly = [(cx + x * s, cy + y * s) for x, y in pts]
    pygame.draw.polygon(screen, (130, 134, 142), poly)
    facet = [(cx - 0.30 * s, cy - 0.30 * s), (cx + 0.05 * s, cy - 0.45 * s),
            (cx + 0.10 * s, cy - 0.05 * s), (cx - 0.20 * s, cy + 0.05 * s)]
    pygame.draw.polygon(screen, (165, 169, 177), facet)
    pygame.draw.polygon(screen, (80, 84, 92), poly, 3)
    pygame.draw.line(screen, (90, 94, 102), (cx + 0.10 * s, cy - 0.05 * s), (cx + 0.25

def draw_paper(screen, cx, cy, s):
    w, h, fold = 0.80 * s, 0.95 * s, 0.25 * s
    x, y = cx - w / 2, cy - h / 2
    poly = [(x, y), (x + w - fold, y), (x + w, y + fold), (x + w, y + h), (x, y + h)]
    pygame.draw.polygon(screen, (240, 240, 235), poly)
    corner = [(x + w - fold, y), (x + w - fold, y + fold), (x + w, y + fold)]
    pygame.draw.polygon(screen, (205, 205, 215), corner)
    pygame.draw.polygon(screen, (150, 150, 160), corner, 2)
    pygame.draw.polygon(screen, (150, 150, 160), poly, 2)
    for i in range(4):
        ly = y + fold + 0.12 * s + i * 0.15 * s
        pygame.draw.line(screen, (170, 175, 190), (x + 0.12 * s, ly), (x + w - 0.12 * s

def draw_scissors(screen, cx, cy, s):
    # finger loops
    for sign in (-1, 1):
        pygame.draw.circle(screen, (200, 60, 60), (int(cx + sign * 0.25 * s), int(cy +
    # blades cross at the pivot
    blade = (205, 210, 220)
    pygame.draw.line(screen, blade, (cx - 0.20 * s, cy + 0.20 * s), (cx + 0.28 * s, cy
    pygame.draw.line(screen, blade, (cx + 0.20 * s, cy + 0.20 * s), (cx - 0.28 * s, cy
    pygame.draw.circle(screen, (90, 95, 105), (int(cx), int(cy - 0.07 * s)), 5)

ICON_DRAWERS = {"ROCK": draw_rock, "PAPER": draw_paper, "SCISSORS": draw_scissors}

```

3. In `__init__` (add after `self.font_arena = ...`, and after `self.showing_result = False`):

python

```
self.font_count = pygame.font.SysFont(None, 72)
```

python

```
self.animating = False
self.anim_start = 0
self.pending_player = None
self.pending_cpu = None
```

4. Replace `play_round` with these two methods. `play_round` now only starts the animation, and `resolve_round` does the scoring at the reveal:

python

```

def play_round(self, choice):
    self.pending_player = choice
    self.pending_cpu = self.choose_cpu_move()
    self.history.append(choice)
    self.history = self.history[-HISTORY_LEN:]

    self.player_choice = None
    self.cpu_choice = None
    self.result_text = "Get ready..."
    self.result_color = (190, 195, 205)
    self.showing_result = False
    self.animating = True
    self.anim_start = pygame.time.get_ticks()

def resolve_round(self):
    self.animating = False
    self.player_choice = self.pending_player
    self.cpu_choice = self.pending_cpu

    outcome = self.determine_winner(self.player_choice, self.cpu_choice)
    if outcome == "PLAYER":
        self.player_score += 1
        self.result_text = f"You Win! {self.player_choice} beats {self.cpu_choice}."
        self.result_color = (80, 230, 120)
    elif outcome == "CPU":
        self.cpu_score += 1
        self.result_text = f"You Lose! {self.cpu_choice} beats {self.player_choice}"
        self.result_color = (240, 80, 80)
    else:
        self.result_text = f"It's a Draw! Both picked {self.player_choice}."
        self.result_color = (240, 210, 80)

    self.showing_result = True
    self.round_resolved_time = pygame.time.get_ticks() # 1.8s timer starts at the

    if self.player_score >= WIN_SCORE or self.cpu_score >= WIN_SCORE:
        self.match_over = True
        self.winner_text = "YOU WIN THE MATCH!" if self.player_score >= WIN_SCORE e

```

5. In `handle_event` (add right after the `if self.match_over:` block):

python

```

    if self.animating:
        return

```

6. Replace `update`:

python

```

def update(self):
    if self.match_over:
        return
    now = pygame.time.get_ticks()
    if self.animating:
        if now - self.anim_start >= ANIM_DURATION:
            self.resolve_round()
        return
    if self.showing_result and (now - self.round_resolved_time >= self.display_dura
        self.player_choice = None
        self.cpu_choice = None
        self.result_text = "Make your move!"
        self.result_color = (190, 195, 205)
        self.showing_result = False

```

7. In `reset` (add one line):

python

```

self.animating = False

```

8. In `render`, replace everything from `p_str = ...` through the two `screen.blit(arena,...)` lines:

python

```

now = pygame.time.get_ticks()
icon_cy, icon_size = 160, 70
slots = (
    (self.width // 4, "Your Pick", self.player_choice),
    (3 * self.width // 4, "CPU Pick", self.cpu_choice),
)
for cx, label, choice in slots:
    lbl = self.font_hud.render(label, True, (225, 225, 230))
    screen.blit(lbl, (cx - lbl.get_width() // 2, 92))
    if self.animating:
        t = now - self.anim_start
        bob = -abs(math.sin(math.pi * t / COUNTDOWN_STEP)) * 14 # one fist pu
        draw_rock(screen, cx, icon_cy + bob, icon_size)
    elif choice:
        ICON_DRAWERS[choice](screen, cx, icon_cy, icon_size)
    else:
        dash = self.font_arena.render("--", True, (110, 116, 130))
        screen.blit(dash, (cx - dash.get_width() // 2, icon_cy - dash.get_height() // 2))

if self.animating:
    step = 3 - min(2, (now - self.anim_start) // COUNTDOWN_STEP)
    num = self.font_count.render(str(step), True, (245, 245, 245))
    screen.blit(num, (self.width // 2 - num.get_width() // 2, icon_cy - num.get_height() // 2))

```

Notes:

- The animation is a 3-2-1 countdown with both sides showing a fist that pumps once per count (1.2s total). Both picks stay hidden until the reveal.
- The CPU's move is chosen at click time, so the adaptive logic is unchanged. Scoring moved to `resolve_round`, so the score never changes before the reveal. The match-over check moved there too, so the winner banner appears only after the final reveal.
- Clicks are ignored while `animating` is true. The 1.8s result timer starts inside `resolve_round`, so the animation never eats into it.
- The icons are drawn relative to a size value, so you can scale them by changing `icon_size`. The positions assume a window roughly 500px or wider, which fits your button row. If your window is much narrower, adjust the `width // 4` offsets.

